Chapter 16 — Concurrency and Async

What you will learn. Concurrency vs parallelism; I/O-bound work (time spent waiting on network, disk, database) vs CPU-bound work (time spent computing); Python's four tools for doing several things at once — **threads** (`threading`, locks, `queue.Queue`), **processes** (`multiprocessing`), **executors** (`concurrent.futures`) and **async/await** (`asyncio`: event loop, coroutines, tasks, `gather`, `TaskGroup`, timeouts, async iteration). What the **GIL** is, why it exists, and where **free-threaded** (no-GIL) CPython stands in 3.13 and 3.14. A "which tool?" table and the pitfalls: blocking calls in async code, race conditions, deadlocks.

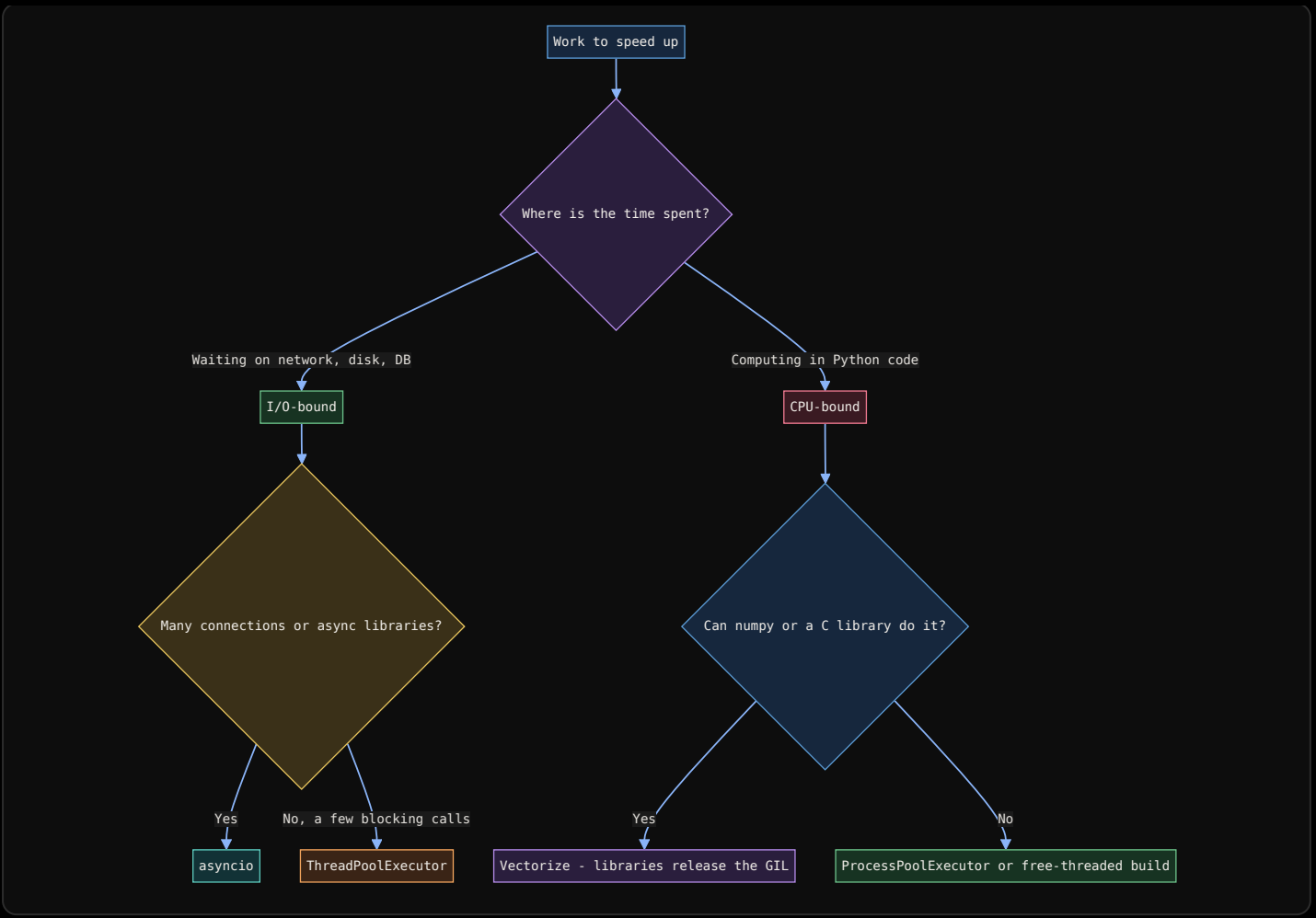
Prerequisites. Functions and closures (Ch 7), generators (Ch 11), context managers (Ch 12).

Code. `chapters/16-concurrency-and-async/code/` — `threads_basics.py`, `gil_benchmark.py` (makes the plot in `img/`), `multiprocessing_demo.py`, `futures_demo.py`, `asyncio_demo.py`, `pitfalls_demo.py`. Outputs are from Python 3.14.7 (default build, GIL on) on a 14-core Mac; timings differ on your machine, the *shape* does not.

Java side. General theory — memory models (when one thread's writes become visible to another), `synchronized`, `ReentrantLock`, semaphores, latches, atomics, concurrent collections, deadlock/livelock (threads keep reacting to each other without progress)/starvation (a thread never gets its turn) and classic LLD (low-level design) problems — is in LLD Handbook Chapter 12 — Concurrency concepts. This chapter covers what is specific to Python.

Words used everywhere in this chapter (defined once):

Term	Meaning
Process	A running program with its own memory space. Two processes do not see each other's variables.
Thread	A path of execution *inside* a process. Threads of one process share its memory.
Task (asyncio)	A coroutine (a function that can pause and resume) scheduled on an event loop (the scheduler that runs whichever task is ready next). Many tasks share one thread.
Blocking call	A call that makes the caller wait (reading a socket, `time.sleep`, a disk read).
Race condition	A bug where the result depends on the unlucky timing of two threads touching shared data.
Critical section	Code that must be run by only one thread at a time.
Lock / mutex	An object that lets only one holder into a critical section ("mutex" = *mut*ual *ex*clusion).
Deadlock	Two or more threads each waiting forever for a lock the other holds.
GIL	Global Interpreter Lock: in standard CPython only one thread runs Python bytecode at a time.



16.1 Concurrency vs parallelism

What it is

- **Concurrency** = *dealing with* many things at once: several tasks in progress, steps interleaved. Works on a single CPU core.
- **Parallelism** = *doing* many things at the same instant on different cores.

One chef boiling pasta and chopping meanwhile is concurrency; three chefs is parallelism. Concurrency is about *structure* (never idle while waiting); parallelism about *execution* (more hands). Parallelism is a kind of concurrency, not vice versa.

Why it matters

Threads on a CPU-heavy pure-Python loop in standard CPython do nothing (the GIL); threads on a program spending 95 % of its time waiting for HTTP responses can make it 10–50× faster.

How it works

	Concurrency	Parallelism
Needs multiple cores?	No	Yes
Goal	Stay busy while waiting	Finish computation sooner
Python tools	threads, asyncio	multiprocessing, free-threaded threads, C extensions that release the GIL
Typical workload	Web servers, scrapers, chat apps	Image processing, simulations, number crunching

---

16.2 I/O-bound vs CPU-bound

What it is

- **I/O-bound**: mostly *waits* (network, disk, DB, user); CPU idle. E.g. downloading 100 pages.
- **CPU-bound**: mostly *computes*; CPU at 100 %. E.g. SHA-256 of a billion strings in a Python loop.

How to tell (3 quick checks)

1. **CPU usage** (`top`, Activity Monitor): ~100 % of one core → CPU-bound; low CPU but slow → I/O-bound.
2. **Profile** (Ch 17): top functions `socket.recv`, `select`, `read` → I/O; your own loops → CPU.
3. **Wall vs CPU time**: `time.perf_counter()` vs `time.process_time()`. Wall ≫ CPU → waiting.

Measured

`gil_benchmark.py` runs 4 CPU tasks (pure-Python loop, 15 million iterations) and 4 I/O tasks (`time.sleep(0.5)`, standing in for a network wait) three ways:

```python
def cpu_task(n: int = 15_000_000) -> int:
    total = 0
    for i in range(n):                    # pure-Python loop: holds the GIL the whole time
        total += i * i
    return total

def io_task(_: int = 0) -> None:
    time.sleep(0.5)                       # sleeping releases the GIL, like waiting on a socket
```

Output:

```
Python 3.14.7, GIL enabled: True
CPU-bound: sequential 1.94s  threads 1.83s  processes 0.87s
I/O-bound: sequential 2.02s  threads 0.51s  processes 0.92s
saved gil_benchmark.png
```

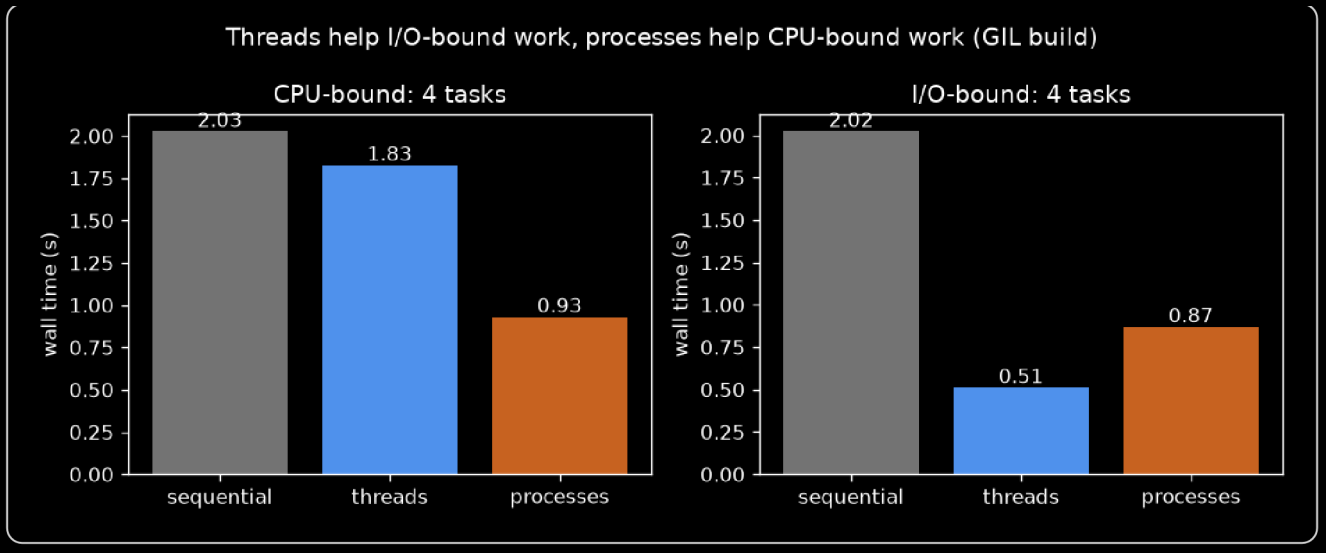


Figure — CPU-bound work only speeds up with processes; I/O-bound work speeds up with threads (processes also help but pay ~0.3–0.4 s start-up).

- **CPU + threads ≈ sequential** (1.83 vs 1.94 s is noise): one thread runs Python at a time.
- **CPU + processes ≈ 2.2× faster**: each process has its own interpreter and GIL. Not 4× because starting processes (`spawn` re-imports the module) and returning data cost time.
- **I/O + threads ≈ 4× faster**: a sleeping thread releases the GIL, so the four waits overlap.

Rule: measure first. Waiting → threads or asyncio. Computing → processes, vectorized libraries, or the free-threaded build.

16.3 Threads with `threading`

What it is

`threading` starts extra threads in your process. Threads share globals and heap objects; each has its own call stack.

How it works

1. `t = threading.Thread(target=func, args=(...), name="T1")`.
2. `t.start()` — the OS starts a thread that calls `func(*args)`.
3. `t.join()` — wait until `t` finishes (optionally `join(timeout)`).
4. A **daemon** thread (`daemon=True`) is killed abruptly when the main program exits; a normal one keeps the program alive. Daemons only for losable work (a heartbeat), never for writing files.

```
def worker(name: str, delay: float, results: dict) -> None:
    time.sleep(delay)                    # stands in for waiting on I/O
    results[name] = threading.current_thread().name

threads = [threading.Thread(target=worker, args=(f"job{i}", 0.2, results), name=f"T{i}")
           for i in range(3)]
for t in threads: t.start()              # all three begin waiting at once
for t in threads: t.join()               # wait for all to finish
```

Output (`threads_basics.py`, line 1):

```
3 x 0.2 s sleeps in threads took 0.21 s -> {'job0': 'T0', 'job1': 'T1', 'job2': 'T2'}
```

The waits overlapped. `target`'s return value is discarded; get results via a shared structure (as here), a `queue.Queue`, or — better — `ThreadPoolExecutor` (§16.8), which returns values and exceptions. Subclassing `Thread` and overriding `run()` also works, but `target` is simpler and more common.

Per-thread data. `threading.local()` gives an object whose attributes are separate per thread (a DB connection, the current request ID). In asyncio, where tasks share a thread, use `contextvars.ContextVar` — each task sees its own value (how web frameworks store the current request).

16.4 Race conditions and locks

What it is

Two threads read-modify-write the same data with interleaved steps. Even `counter += 1` is *load → add → store*: if A loads 5, then B loads 5, both store 6 — one increment lost. A **lock** (`threading.Lock`) admits one holder at a time, turning the read-modify-write into a **critical section**.

The race, demonstrated

```
def unsafe_increment(n: int) -> None:
    global counter
    for _ in range(n):
        current = counter                # read
        time.sleep(0)                    # yield: lets another thread run (widens the window)
        counter = current + 1            # write back a stale value

lock = threading.Lock()

def safe_increment(n: int) -> None:
    global counter
    for _ in range(n):
        with lock:                       # only one thread inside at a time
            current = counter
            time.sleep(0)
            counter = current + 1
```

Output (4 threads × 1,000 increments):

```
unsafe counter (expected 4000): 1000
safe counter   (expected 4000): 4000
```

The unsafe number varies (other runs: 2000, 1031). `time.sleep(0)` only makes the bug show every time; without it the bug appears rarely — worse, because it passes tests and fails in production.

How it works — the synchronization toolbox

Primitive	What it does	Typical use
`Lock`	One holder at a time. Not re-entrant: the same thread acquiring twice blocks itself.	Protect shared state
`RLock`	Re-entrant lock: the owning thread may acquire it again (must release as many times).	Recursive code that re-enters a locked method
`Semaphore(n)`	Up to *n* holders at once.	Limit concurrent DB connections to 10
`Event`	A flag threads can `wait()` on until someone calls `set()`.	"Start now" / "shut down" signal
`Condition`	Wait until a condition becomes true, woken by `notify()`.	Building your own queues
`Barrier(n)`	Blocks until *n* threads arrive, then releases all.	Phased simulations
`queue.Queue`	Thread-safe FIFO with blocking `get`/`put`.	Producer–consumer (next section)

Use locks via `with lock:` — released even on exceptions. Manual `acquire()`/`release()` only for timeouts: `if lock.acquire(timeout=1): ...`.

Common mistakes

- "The GIL makes my code thread-safe." It protects the *interpreter's* internals (a `list` is never corrupted), not *your* multi-step logic: `if key not in d: d[key] = compute()` can still run twice.
- Protecting writes but not reads of a multi-field invariant (e.g. reading `balance` mid-transfer).
- Holding a lock during slow I/O — everyone queues behind it.

In the interview

> **30-second answer:** "A race condition is when the outcome depends on thread timing, usually from an unprotected read-modify-write. Even `x += 1` is several bytecodes, so it's not atomic (indivisible). I protect shared state with a `threading.Lock` used as a context manager, keep critical sections small, or avoid sharing by passing messages through `queue.Queue`."

- *Is `list.append` thread-safe?* — One `append` on a built-in list is safe in CPython (GIL; the free-threaded build uses per-object locks for built-ins), but sequences of operations are not. Don't rely on it for logic.
- *Lock vs RLock?* — RLock lets the same thread re-acquire; Lock would deadlock on itself.
- *Which primitive limits concurrency to N?* — `Semaphore(N)`.

16.5 Producer–consumer with `queue.Queue`

What it is

Producer–consumer splits work into threads that *make* items and threads that *process* them, joined by a queue. `queue.Queue` is a thread-safe FIFO with all locking built in.

Why it matters

Threads never touch each other's data — only `put` and `get`. A **bounded** queue (`maxsize=N`) adds **backpressure**: a fast producer blocks on `put` instead of filling memory. Queues and backpressure are core system-design ideas too (System Design Fundamentals).

How it works

1. `q = queue.Queue(maxsize=5)`.
2. Producers `q.put(item)` (blocks while full).
3. Consumers `q.get()` (blocks while empty), process, `q.task_done()`.
4. To stop, each producer puts a **sentinel** ("no more work", here `None`); the consumer stops after one per producer. (3.13 added `Queue.shutdown()` for this.)

```
SENTINEL = None

def producer(q, items):
    for item in items:
        q.put(item)                    # blocks if the queue is full (backpressure)
    q.put(SENTINEL)

def consumer(q, out, n_producers):
    finished = 0
    while finished < n_producers:
        item = q.get()                 # blocks until an item is available
        if item is SENTINEL:
            finished += 1
        else:
            out.append(item * item)
        q.task_done()
```

Output:

```
consumer squared 10 items, sorted: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

`q.join()` (unused here) blocks until every `put` is matched by a `task_done()` — "wait for all work processed" without stopping workers.

Queue class	Order
`queue.Queue`	FIFO
`queue.LifoQueue`	Stack (last in, first out)
`queue.PriorityQueue`	Smallest item first (put `(priority, item)` tuples)
`queue.SimpleQueue`	Unbounded FIFO, no `task_done` / `join`
`multiprocessing.Queue`	Between processes (items are pickled)
`asyncio.Queue`	Between asyncio tasks (not thread-safe)

Real-world examples

Web crawlers (fetcher → parser threads), logging (`logging.handlers.QueueHandler` + `QueueListener` move slow log I/O off request threads), PyTorch's `DataLoader` (worker *processes* feeding batches through queues).

16.6 The GIL (Global Interpreter Lock)

What it is

In standard CPython the **GIL** is one lock a thread must hold to execute bytecode, so **only one thread per process runs Python code at any instant**, even on 64 cores.

Why it exists

CPython manages memory with **reference counts** (Ch 17). Updating them from several threads would need a lock per object or atomic CPU instructions everywhere — slower and much harder to get right. One big lock kept the interpreter simple, single-threaded code fast, and C extensions easy to write — a big reason numpy and the scientific stack exist.

How it works

1. A thread holds the GIL while running bytecode.
2. It **releases** it during blocking I/O (`socket.recv`, file reads, `time.sleep`) and when a C extension releases it explicitly (numpy on large arrays, `hashlib` on big buffers, `zlib`, many image and DB libraries).
3. A waiting thread asks; after the **switch interval** (`sys.getswitchinterval()`, default 0.005 s = 5 ms) the holder is asked to drop it.

Workload	Threads with the GIL
I/O-bound	Scale well — the GIL is released while waiting
CPU-bound pure Python	No speed-up; can even be slightly slower (switching overhead)
CPU-bound in numpy / C that releases the GIL	Can scale across cores

Free-threaded CPython (PEP 703) — status

- **Python 3.13 (Oct 2024):** an *optional, experimental* GIL-disabled build ships ("free-threaded"; executable `python3.13t`). PEP 703 is the design.
- **Python 3.14 (Oct 2025):** per **PEP 779**, **officially supported** (no longer experimental) but **still optional — not the default**: `python3.14` has a GIL, `python3.14t` does not. Single-threaded slowdown roughly 5–10 % (platform/compiler dependent); memory use is higher.

- **Extensions:** importing a C extension not marked free-threading-compatible **re-enables the GIL** (with a warning); force it with `PYTHON_GIL=1` or `python -X gil=1`. Many major packages (numpy among them) publish free-threaded wheels (pre-built `.whl` packages) — check before relying on it.
- **Check at runtime:**

```
import sys, sysconfig
sysconfig.get_config_var("Py_GIL_DISABLED")  # 1 on a free-threaded build, else 0/None
sys._is_gil_enabled()                        # False when running without the GIL (3.13+)
```

- **What changes for you:** CPU-bound threads can scale across cores — but races become *more* likely, so §16.4's rules still apply. Built-in `list`/`dict`/`set` use per-object locks so they aren't corrupted; multi-step logic still needs your locks.

Other ways around the GIL: **processes** (one GIL each); **subinterpreters** (independent interpreters in one process) — own GIL each since 3.12 (PEP 684), and 3.14 adds `concurrent.interpreters` and `InterpreterPoolExecutor` (PEP 734); **native code** that releases the GIL (numpy, Cython — Python-like language compiled to C, Rust via PyO3).

Common mistakes

- "Python can't do concurrency." It does I/O concurrency very well; the GIL limits *parallel bytecode*.
- "The GIL makes code thread-safe." See §16.4.
- Assuming all implementations match: Jython (JVM) and IronPython (.NET) have no GIL; PyPy (JIT compiler) has one.

In the interview

30-second answer: "The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time. It exists because reference counting isn't thread-safe without it, and it keeps single-threaded code and C extensions simple. It's released during blocking I/O and by many C extensions, so threads are fine for I/O-bound work; for CPU-bound Python I use multiprocessing or push work into numpy. Since 3.13 there's an optional free-threaded build, officially supported in 3.14 but not the default."

- *Does the GIL make `x += 1` atomic?* — No; a switch can happen between load and store.
- *Why doesn't numpy suffer from the GIL?* — Its heavy loops run in C and release it.
- *Switch to the free-threaded build today?* — Only after checking every C-extension dependency supports it and benchmarking; single-threaded code gets a bit slower.

16.7 `multiprocessing`

What it is

Runs functions in separate **processes**, each with its own interpreter, memory and GIL — true parallelism for CPU-bound Python. The API mirrors `threading`.

How it works

1. **Start method** — how the child is created:

Method	How	Default on
`spawn`	Fresh interpreter; re-imports your main module; arguments are pickled	Windows, macOS
`fork`	Copies the parent process (fast, but unsafe if the parent has threads)	— (was Linux's default up to 3.13)
`forkserver`	Forks from a clean single-threaded server process	Linux and other POSIX (Unix-like) systems except macOS, from **3.14**

`spawn` / `forkserver` re-import your script, so **always guard start-up code with `if __name__ == "__main__":`**, or each child starts children of its own.

2. **Pickling** — arguments and results travel via `pickle` (object ↔ bytes). Lambdas, nested functions and open sockets can't be pickled; use top-level functions.
3. **No shared memory by default** — each process has its own globals (and its own pid, the OS process ID).

```
counter = 0                              # each process gets its OWN copy

def bump_global(n: int, parent_pid: int) -> None:
    global counter
    for _ in range(n):
        counter += 1
    print(f"  child: different pid from parent={os.getpid() != parent_pid}, sees counter={counter}")

def bump_shared(shared, n: int) -> None:
    for _ in range(n):
        with shared.get_lock():          # Value comes with its own lock
            shared.value += 1

if __name__ == "__main__":
    p = mp.Process(target=bump_global, args=(1000, os.getpid()))
    p.start(); p.join()
    shared = mp.Value("i", 0)            # 'i' = C int in shared memory
    # ... start 4 processes running bump_shared(shared, 1000) ...
    with mp.Pool(processes=4) as pool:
        pool.map(square, range(10))
```

Output (`multiprocessing_demo.py`):

```
start method: spawn, CPUs: 14
  child: different pid from parent=True, sees counter=1000
parent counter after child ran: 0  (not shared!)  exitcode=0
shared Value after 4 x 1000 increments: 4000
pool.map squares: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
pool.map chunksize=3: [0, 1, 4, 9, 16] ...
lambda in Pool -> PicklingError
```

Shared-state options and caveats

Tool	What it is	Caveat
Return values / `Queue` / `Pipe`	Message passing (pickled)	Preferred; cost grows with data size
`Value`, `Array`	A C number/array in shared memory	Needs `get_lock()` for read-modify-write
`multiprocessing.shared_memory.SharedMemory`	Raw shared bytes (wrap in a numpy array)	You manage layout, lifetime and `unlink()`
`Manager()`	A server process holding a dict/list that proxies talk to	Easy but slow (every access is IPC — inter-process communication)

`Pool` methods: `map` (ordered, blocks), `imap` / `imap_unordered` (lazy, streamed), `starmap` (multiple arguments), `apply_async` (one call → `AsyncResult`). `chunksize` batches items per message — big chunks cut overhead for many small tasks.

When not to use it

- Tiny tasks — pickling and start-up (tens to hundreds of ms under `spawn`) outweigh the work.
- Huge inputs copied to every worker — use shared memory or let workers load their own slice.
- I/O-bound work — threads or asyncio are lighter.

Common mistakes

- Missing `__main__` guard → `RuntimeError` or runaway process creation under `spawn`.
- Expecting a child's global change to appear in the parent.
- Lambdas/local functions → `PicklingError` / `AttributeError`.
- `fork` with threads → the child may inherit a lock held by a thread that no longer exists, and hang.

Real-world examples

Gunicorn and uWSGI (production Python web servers) run several **worker processes** of a Django/Flask app to use all cores; Celery's default worker pool is "prefork" (processes started in advance); PyTorch `DataLoader(num_workers=4)` loads batches in worker processes.

16.8 `concurrent.futures` — executors

What it is

One high-level API for "run these calls in the background and give me the results": `ThreadPoolExecutor` and `ProcessPoolExecutor` share the interface. You get a **Future** — a result that will exist later.

How it works

1. `with ThreadPoolExecutor(max_workers=8) as ex:` — leaving the block waits for all work (`shutdown(wait=True)`).
2. `fut = ex.submit(fn, *args)`; `fut.result(timeout=None)` returns the value **or re-raises the worker's exception**.
3. `ex.map(fn, iterable)` → results in **input order**, lazily.
4. `as_completed(futures)` → futures in **completion order**.
5. `wait(futures, return_when=FIRST_COMPLETED | FIRST_EXCEPTION | ALL_COMPLETED)` → `(done, pending)`.
6. `fut.cancel()` works only before the call starts.

Default pool sizes (3.13+): threads `min(32, os.process_cpu_count() + 4)`; processes `os.process_cpu_count()`.

```
with ThreadPoolExecutor(max_workers=3) as ex:
    futures = {ex.submit(fake_download, u): u for u in urls}
    for fut in as_completed(futures):
        try:
            print(f"done {futures[fut]} -> {fut.result()}")
        except ConnectionError as e:
            print(f"failed {futures[fut]} -> {e!r}")

with ThreadPoolExecutor(max_workers=3) as ex:
    it = ex.map(fake_download, urls)
    try:
        for r in it:
            print(f"map result {r}")
    except ConnectionError as e:
        print(f"map raised {e!r}")
```

Output (`futures_demo.py`; delays a=0.3 s, b=0.1 s, c=0.2 s, c fails):

```
done b.com -> ('b.com', 100)
failed c.com -> ConnectionError('c.com timed out')
done a.com -> ('a.com', 300)
map result ('a.com', 300)
map result ('b.com', 100)
map raised ConnectionError('c.com timed out')
first completed: 1 done, 1 cancelled
primes: [True, True, True, False] in 0.32s with processes
```

`as_completed` returns the fastest first; `map` keeps input order and raises on *reaching* the failed item; with one worker and three 0.2 s jobs, after the first finishes the second is already running (uncancellable), so only the third is cancelled; switching to processes for CPU work is a one-word change.

Trade-offs

	Raw `threading` / `multiprocessing`	`concurrent.futures`
Return values & exceptions	Manual (queues, shared dicts)	Built in (`Future.result()`)
Thread ↔ process switch	Rewrite	Change the class name
Fine control (daemons, custom sync)	Yes	Limited
Best for	Long-lived workers, pipelines	"Run N calls, collect results"

In the interview

30-second answer: "`concurrent.futures` gives one API over thread and process pools. `submit` returns a Future; `result()` returns the value or re-raises the worker's exception. `map` preserves order, `as_completed` yields in finish order. `ThreadPoolExecutor` for blocking I/O, `ProcessPoolExecutor` for CPU-bound pure Python."

- *Exception in a worker?* — Stored in the Future; raised on `result()` (or iterating `map`). Never checked → silently lost.
- *Timeout?* — `fut.result(timeout=5)` or `as_completed(fs, timeout=5)` → `TimeoutError`.

16.9 `asyncio` — async/await

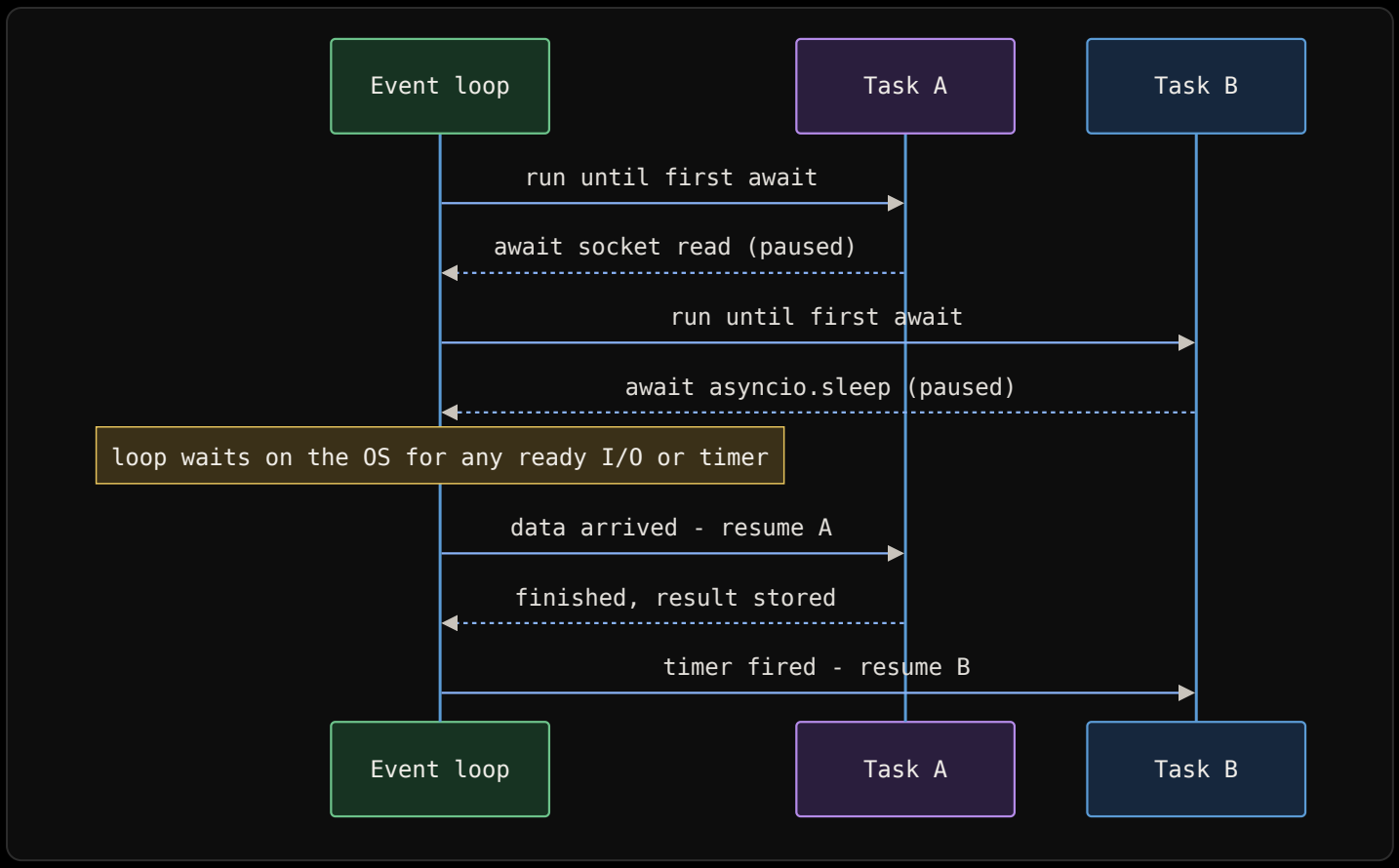
What it is

`asyncio` runs many tasks concurrently in **one thread** with an **event loop**. A **coroutine** (`async def`) pauses at each `await`, letting the loop run something else while it waits for I/O. Nothing is parallel; the speed comes from never sitting idle (one waiter serving 20 tables).

Why it matters

A thread costs stack memory and OS scheduling; 10,000 threads hurt, 10,000 asyncio tasks waiting on sockets are normal. Switches happen only at `await`, so there are far fewer race windows than with threads.

How it works



1. `asyncio.run(main())` creates a loop, runs `main()` to completion, closes it. Call once, at the top.
2. Calling `fetch()` creates a **coroutine object**; nothing runs until it's awaited or wrapped in a task.
3. `await x` pauses until `x` (coroutine, Task or Future) is done.
4. `asyncio.create_task(coro)` schedules it to run **concurrently**, returning a `Task`.
5. The loop uses OS readiness APIs (epoll on Linux, kqueue on macOS/BSD — report which of many connections have data) to resume tasks whose sockets/timers are ready.

Core API with real output (`asyncio_demo.py`)

```python
async def fetch(name: str, delay: float) -> str:
    await asyncio.sleep(delay)           # non-blocking wait
    return f"{name}:{delay}"

await fetch("a", 0.2); await fetch("b", 0.2)                    # sequential: 0.4 s
results = await asyncio.gather(fetch("a", 0.2), fetch("b", 0.2), fetch("c", 0.1))  # 0.2 s

task = asyncio.create_task(fetch("bg", 0.2))    # starts running now
await asyncio.sleep(0.1)
print(task.done())                              # False
value = await task
```

```
type before await: coroutine
awaited: x:0
[0.40s] two sequential awaits
[0.20s] gather -> ['a:0.2', 'b:0.2', 'c:0.1']  (order = argument order)
[0.10s] task done yet? False
[0.20s] task result bg:0.2
```

Key lesson: `await a(); await b()` is **sequential**; concurrency needs `gather`, `create_task` or a `TaskGroup`.

`gather` vs `TaskGroup` (3.11+)

```
res = await asyncio.gather(fetch("ok", 0.1), failing(0.05), return_exceptions=True)

try:
    async with asyncio.TaskGroup() as tg:
        slow = tg.create_task(fetch("slow", 1.0))
        tg.create_task(failing(0.1))
except* ValueError as eg:                       # except* handles an ExceptionGroup
    print(eg.exceptions, slow.cancelled())
```

An `ExceptionGroup` (3.11+) bundles several exceptions raised together (e.g. several failed tasks); `except*` handles each kind inside it.

```
gather with return_exceptions -> ['ok:0.1', ValueError('boom')]
[0.10s] TaskGroup raised (ValueError('boom'),); slow cancelled=True
```

	`asyncio.gather`	`asyncio.TaskGroup`
On first failure	Raises it, but the **other tasks keep running** unattended	**Cancels** the siblings, waits for them, raises an `ExceptionGroup`
Collect errors as values	`return_exceptions=True`	Catch with `except*`
Results	List in argument order	`task.result()` on each task
Recommended for new code	Simple fan-out (start many, collect results)	Yes — "structured concurrency": no task outlives its block

Timeouts, limits and async iteration

```
async with asyncio.timeout(0.1):                # 3.11+, can wrap several awaits
    await fetch("too-slow", 1.0)
await asyncio.wait_for(fetch("too-slow", 1.0), timeout=0.1)   # one awaitable

sem = asyncio.Semaphore(2)                      # at most 2 at a time
async def limited(i):
    async with sem:
        await asyncio.sleep(0.1)
        return i

async def ticker(n):                            # async generator
    for i in range(n):
        await asyncio.sleep(0.05)
        yield i

async with AsyncResource():                     # __aenter__ / __aexit__
    squares = [i * i async for i in ticker(4)] # async comprehension
```

```
asyncio.timeout -> TimeoutError
wait_for -> TimeoutError
[0.31s] 6 jobs, 2 at a time -> [0, 1, 2, 3, 4, 5]
[0.00s] open resource
[0.20s] close resource
async comprehension -> [0, 1, 4, 9]
```

On timeout the inner task is **cancelled** (`CancelledError` raised inside it) and the caller sees the built-in `TimeoutError` (since 3.11 `asyncio.TimeoutError` is the same class). Six jobs, limit two → 3 rounds × 0.1 s.

Async protocols: `async for` uses `__aiter__`/`__anext__`; `async with` uses `__aenter__`/`__aexit__`. Also useful: `asyncio.Queue`, `asyncio.Event`, `asyncio.Lock`, `asyncio.as_completed`.

Mixing sync and async

```
def blocking_io() -> str:
    time.sleep(0.3)                 # a sync library call (e.g. requests.get)
    return "sync result"

await asyncio.gather(asyncio.to_thread(blocking_io), asyncio.to_thread(blocking_io))
async def bad():
    return blocking_io()            # time.sleep inside a coroutine blocks EVERYTHING
await asyncio.gather(bad(), bad())
```

```
[0.31s] two blocking calls via to_thread ran together
[0.60s] two blocking calls inside coroutines ran one after another
```

One blocking call in a coroutine freezes the **entire** loop. Use async libraries (`httpx`/`aiohttp` not `requests`, `asyncpg` not sync `psycopg2`, `asyncio.sleep` not `time.sleep`) or push sync calls to a thread with `asyncio.to_thread` (3.9+) / `loop.run_in_executor`. CPU-heavy work → `ProcessPoolExecutor` via `run_in_executor`.

When to use asyncio

- **Yes:** many simultaneous network connections — web servers/APIs (FastAPI, Starlette, aiohttp), websockets (long-lived two-way connections), chat, crawlers, fan-out to many microservices (small independent services forming one application) or LLM APIs.
- **No:** CPU-bound work (one thread!), a script making three HTTP calls (threads are simpler), a codebase whose key libraries are all synchronous.

Common mistakes

- Forgetting `await` — `fetch()` alone only creates a coroutine; Python warns *"coroutine ... was never awaited"*.
- `create_task` without keeping the reference — the loop holds only a weak reference, so the task can be garbage-collected mid-run. Keep tasks in a set or use a `TaskGroup`.
- `asyncio.run()` inside a running loop (e.g. Jupyter) → `RuntimeError`; in notebooks just `await main()`.
- Catching `CancelledError` without re-raising — breaks timeouts and TaskGroup cancellation.
- Sharing `asyncio.Queue`/`Lock` across threads — not thread-safe; use `loop.call_soon_threadsafe` or `asyncio.run_coroutine_threadsafe`.

Real-world examples

FastAPI/Starlette on uvicorn (optional uvloop, a faster loop built on libuv — the C library behind Node.js); Discord bots (`discord.py`), Home Assistant, and the Anthropic and OpenAI Python SDKs' `Async...` clients.

In the interview

30-second answer: "asyncio is single-threaded cooperative concurrency — tasks voluntarily yield at `await` instead of being interrupted. The event loop resumes whichever task's I/O is ready. It scales to thousands of connections cheaply, but anything blocking stalls everything, so I use async libraries or `asyncio.to_thread`. For fan-out I prefer `TaskGroup` over `gather` because a failure cancels the siblings."

- *Is asyncio parallel?* — No, one thread.
- *`gather` vs `TaskGroup`?* — `gather` leaves siblings running on error; `TaskGroup` cancels them and raises an `ExceptionGroup`.
- *Calling sync code from async?* — `await asyncio.to_thread(fn, *args)`.
- *Cap concurrency at 10?* — `asyncio.Semaphore(10)`.

16.10 Choosing the right tool

Situation	Use	Why
A few to hundreds of blocking I/O calls, sync libraries	`ThreadPoolExecutor`	Simple, GIL released during I/O
Thousands of concurrent connections, async libraries	`asyncio`	Tasks are cheap, no thread per connection
CPU-bound pure Python	`ProcessPoolExecutor` / `multiprocessing.Pool`	Each process has its own GIL
CPU-bound numeric work	numpy/pandas (vectorize, Ch 17–18)	C loops, often multi-threaded internally
CPU-bound, want shared memory, deps support it	Free-threaded build (3.14t) + threads	Real parallel threads
Long-running pipeline stages	Threads/processes + `Queue`	Backpressure, clean shutdown
Async app needs a sync library	`asyncio.to_thread`	Keeps the loop responsive
Async app needs heavy CPU	`loop.run_in_executor(ProcessPoolExecutor(), ...)`	Moves CPU work off the loop
Scale beyond one machine	Celery, RQ, Ray, Dask, Spark	Distributed task queues/clusters across many machines

	Threads	Processes	asyncio
Parallel CPU (GIL build)	No	Yes	No
Memory per unit	~MB stack (virtual)	Whole interpreter (tens of MB)	~KB per task
Shared memory	Yes (needs locks)	No (pickle/IPC)	Yes (single thread)
Switching	Pre-emptive (the OS can interrupt at any time)	OS	Cooperative (at `await`)
Race risk	High	Low	Low
Works with sync libs	Yes	Yes	Needs `to_thread`

16.11 Common pitfalls

Blocking calls inside async code

`time.sleep`, `requests.get`, a sync DB driver, big file reads or a long CPU loop in a coroutine halts every task (§16.9). Diagnose with **debug mode** (`asyncio.run(main(), debug=True)` or `PYTHONASYNCIODEBUG=1`), which logs callbacks slower than 100 ms. Python 3.14 adds `python -m asyncio ps PID` / `pstree PID` to inspect a running process's tasks.

Deadlocks

Thread 1 holds lock A and waits for B; thread 2 holds B and waits for A — both wait forever.

```
def transfer(first, second, name, log):
    with first:
        time.sleep(0.05)                              # make the bad interleaving likely
        if second.acquire(timeout=0.5):               # a real deadlock would wait forever
            second.release()
            log.append(f"{name} ok")
        else:
            log.append(f"{name} gave up: DEADLOCK")

run((lock_a, lock_b), (lock_b, lock_a))   # opposite order
run((lock_a, lock_b), (lock_a, lock_b))   # same order
```

Output (`pitfalls_demo.py`):

```
opposite order : ['T1 gave up: DEADLOCK', 'T2 ok']
same order     : ['T1 ok', 'T2 ok']
RLock re-entered by the same thread: fine
plain Lock re-acquire (timeout 0.1): False
```

The timeout lets the demo finish. Fixes:

1. **Global lock ordering** — always acquire in one order (e.g. by account id).
2. **Fewer locks** — one coarse lock, or no shared state (queues, immutable data).
3. **Timeouts** on `acquire` to detect and back off.
4. `RLock` when the same thread legitimately re-enters (a plain `Lock` deadlocks on itself — last line).
5. asyncio: a task awaiting something only it could produce, or a TaskGroup child waiting on its parent — same cure (clear ownership).

Other pitfalls

Pitfall	Symptom	Fix
Unprotected shared state	Wrong totals, rare	Lock, or message passing
Unhandled exception in a `Thread`	Printed to stderr, thread dies silently	Use executors and call `result()`; or set `threading.excepthook`
Too many threads	Memory, context-switch overhead (saving one thread's state, loading another's)	Bounded pool
Missing `__main__` guard	Recursive spawning / `RuntimeError`	Guard the entry point
Huge arguments to processes	Slow pickling	Shared memory, or load inside the worker
Forgotten `await` / lost task reference	Work silently never runs / vanishes	Linters (ruff), `TaskGroup`
`fork` after threads started	Random hangs in child	`spawn` / `forkserver` (defaults on macOS/Windows, and Linux from 3.14)

In the interview

- *Four conditions for deadlock?* — Mutual exclusion (one holder per resource), hold-and-wait (holding one lock while waiting for another), no pre-emption (locks can't be taken away), circular wait. Break any one (lock ordering breaks circular wait). Details: LLD Ch 12.
- *Find a hung thread?* — `faulthandler.dump_traceback_later()` or `py-spy dump --pid` show every thread's stack.

16.12 Chapter self-check

1. Concurrency vs parallelism in one line each? → Overlapping waits vs simultaneous execution on several cores.
2. Why do threads speed up downloads but not a pure-Python prime search? → I/O releases the GIL; bytecode execution doesn't.
3. Why is `counter += 1` unsafe across threads, and what does `with lock:` guarantee on an exception? → Load, add, store can interleave; the lock is released.
4. What does `q.task_done()` pair with? → `q.join()`, which waits until every item is marked done.
5. Default switch interval; how to check if the GIL is on? → 5 ms; `sys._is_gil_enabled()`.
6. Two ways to get CPU parallelism with a GIL? → Processes; C extensions that release the GIL (numpy).
7. Why must `multiprocessing` code have `if __name__ == "__main__":`? → `spawn`/`forkserver` re-import the main module in each child.
8. Linux default start method from 3.14? → `forkserver`.
9. `ex.map` vs `as_completed` ordering? → Input order vs completion order.
10. What runs when you call an `async def` function? Two `await`s in a row — concurrent? → Nothing (a coroutine object); no, sequential.
11. What does `asyncio.timeout` raise to the caller, and why is `time.sleep` in a coroutine bad? → `TimeoutError`; it blocks the whole loop.
12. Siblings when one TaskGroup task fails? → Cancelled; an `ExceptionGroup` is raised.
13. Run `requests.get` from async code without freezing the loop? → `await asyncio.to_thread(requests.get, url)`.
14. Status of free-threaded CPython in 3.14? → Officially supported (PEP 779), optional, not the default (`python3.14t`).
15. Simplest way to prevent lock-ordering deadlocks? → Always acquire locks in one global order.

Chapter TL;DR

- **First classify the work**: waiting (I/O-bound) or computing (CPU-bound). Measure.
- **Threads**: cheap concurrency for I/O; share memory → protect with `Lock`, or pass messages through `queue.Queue` (bounded, with sentinels).
- **GIL**: one thread runs Python bytecode at a time in standard CPython; released on I/O and by C extensions. Free-threaded build: experimental in 3.13, supported but optional in 3.14 (`python3.14t`, ~5–10 % single-thread cost).
- **Processes**: real parallelism; no shared globals; arguments are pickled; guard `__main__`; `spawn` on macOS/Windows, `forkserver` on Linux from 3.14.
- **Executors**: `ThreadPoolExecutor`/`ProcessPoolExecutor`, `submit` → `Future.result()`, `map` (ordered), `as_completed` (finish order). The default for "run N things".
- **asyncio**: one thread, event loop, coroutines pause at `await`. `gather`/`TaskGroup`, `asyncio.timeout`, `Semaphore`, async generators/context managers, `to_thread` for sync code. Never block the loop.
- **Pitfalls**: races, deadlocks (lock ordering), blocking calls in async, lost exceptions, missing `__main__` guard.